Enterprise Governance, Risk, Compliance & Procurement Platform

(e-GRCP)

Software Testing Report

Prepared By:	Prajna SR
Role:	Software Testing Intern
Organization:	Enterprise Software Solutions Pvt. Ltd.
Department:	Quality Assurance / Frontend Engineering
Project Type:	Enterprise React Frontend Application
Document Version:	1.0
Date:	July 6, 2026

Contents

1 Introduction

The Enterprise Governance, Risk, Compliance & Procurement Platform (e-GRCP) is a modular, single-page enterprise frontend application built to consolidate governance, risk management, regulatory compliance, procurement, and vendor management workflows into a single, unified interface. The application is developed using React 19 and Vite, with Redux Toolkit managing centralized application state and Redux Persist ensuring session continuity across browser reloads. Communication with data sources is handled through Axios, currently configured against a structured mock JSON data layer that simulates real backend responses.

Given the enterprise nature of the platform – spanning authentication, role-based access control, procurement workflows, vendor records, risk registers, compliance tracking, approvals, audit trails, and executive reporting dashboards – rigorous software testing was essential before the application could be considered stable for demonstration and future backend integration.

The primary purpose of this testing exercise was to independently verify that the application behaves correctly and predictably across its core functional areas. Specifically, testing aimed to confirm that:

- Users are able to authenticate successfully and that session state persists correctly using Redux Persist.
- Role-based route protection correctly restricts or allows access to specific modules based on the logged-in user's assigned role.
- Navigation across procurement, vendor, risk, compliance, approval, audit, and reporting modules functions without broken links or rendering errors.
- Redux Toolkit slices correctly initialize, update, and reflect state changes across connected components, including data loaded from the mock JSON layer.
- Axios-based service calls correctly retrieve and deliver mock data to the UI layer without unhandled errors.
- Form-based screens (such as vendor onboarding, procurement requests, and settings) correctly validate user input using React Hook Form combined with Yup schema validation, rejecting invalid entries and accepting valid ones.
- The application layout remains responsive and visually consistent across common desktop and tablet viewport sizes, using Material UI's responsive grid and component system.

Testing was carried out manually through structured, scenario-based test cases, supplemented by the project's existing automated unit test suite (Jest and React Testing Library) covering authentication, routing guards, Redux slices, and individual module pages. This combination of manual verification and automated regression coverage provided confidence that the platform's frontend logic is functionally sound.

This report documents the scope, environment, methodology, and detailed results of that testing effort, along with a summary of outcomes, issues identified and resolved during development, and an overall conclusion on the readiness of the application.

2 Testing Scope

Testing for the e-GRCP platform focused exclusively on frontend behavior, since the application currently operates against a mock JSON data layer rather than a live backend service. The scope of testing covered the following areas:

- **User Interface:** Verification of layout consistency, Material UI component rendering, theming (light/dark mode), and visual alignment across dashboard, procurement, vendor, risk, compliance, and reporting screens.
- **Navigation:** Verification of React Router DOM v7 routing across all module pages, sidebar/menu links, breadcrumb behavior, and nested route rendering (e.g., vendor list to vendor detail).
- **Authentication:** Verification of the login flow, credential validation, session token handling, and persisted authentication state across page refreshes.
- **Role Protected Routes:** Verification that the custom Route Guard correctly permits or denies access to modules based on the authenticated user's role.
- **Redux State Management:** Verification of Redux Toolkit slices (auth, dashboard, procurement, vendor, risk, compliance, approval, audit, report, notification, UI, counter) for correct initial state, action dispatch, and reducer updates.
- **API Communication using Axios:** Verification of the Axios-based service/API client layer for correctly issuing requests and handling mock JSON responses, including simulated loading and error states.
- **Mock JSON Data Integration:** Verification that mock datasets correctly populate tables, cards, charts, and detail views across all modules.
- **Form Validation:** Verification of React Hook Form and Yup validation rules on input forms, including mandatory field checks, format checks, and error message display.
- **Responsive Layout:** Verification of layout behavior across desktop and tablet screen widths using Material UI's responsive grid system.

Testing did *not* include backend/server-side testing, database testing, load/performance testing, or security penetration testing, as no live backend, database, or external API is currently part of the application architecture.

3 Test Environment

The following table summarizes the environment and technology versions used while performing the testing activities described in this report.

Component	Details
Operating System	Windows 11 (64-bit)
Browser	Google Chrome (latest stable release)
React	React 19
Build Tool	Vite
State Management	Redux Toolkit
State Persistence	Redux Persist
Routing	React Router DOM (v7)
HTTP Client	Axios
UI Component Library	Material UI (MUI)
Data Source	Mock JSON Data (simulated backend responses)

All testing was carried out in a development build served locally via the Vite development server, with the application running in the browser environment listed above. No external or production infrastructure was used during this testing phase.

4 Testing Methodology

Testing was carried out using a structured, manual, scenario-driven approach, supported by the project's existing automated test suite. The methodology applied is summarized below.

4.1 Functional Testing

Each core feature of the application – login, navigation, procurement actions, vendor management, and form submissions – was tested against its expected functional behavior to confirm that the feature performs the task it was designed for, using both valid and invalid inputs where applicable.

4.2 UI Testing

Screens were visually inspected across modules to confirm correct rendering of Material UI components, consistent spacing and alignment, correct theming, and absence of layout-breaking errors.

4.3 Navigation Testing

All primary and nested routes configured in `AppRoutes.jsx` and `router.jsx` were manually traversed to confirm that each route renders the correct component, that protected routes redirect unauthenticated users appropriately, and that browser back/forward navigation behaves as expected.

4.4 Validation Testing

Forms built with React Hook Form and Yup schemas were tested with empty fields, malformed input, and valid input to confirm that validation rules trigger correctly and that appropriate error messages are displayed to the user.

4.5 State Management Testing

Redux Toolkit slices were tested to confirm correct initial state, correct state transitions on dispatched actions, and correct persistence/rehydration of relevant slices via Redux Persist across page reloads. Selected slices were additionally covered by automated unit tests using Jest and React Testing Library.

5 Test Cases

This section documents five representative test cases covering the most critical functional areas of the e-GRCP platform: authentication, access control, navigation, state management, and form validation.

Test Case 1: Login Authentication

Test Case ID	TC-01
Objective	To verify that a user can successfully log in using valid credentials and that the session is correctly established and persisted.
Preconditions	Application is running; user is on the Login page; a valid mock user account exists in the mock data set.
Test Steps	1. Navigate to the login page. 2. Enter a valid username and password. 3. Click the ”Login” button. 4. Refresh the browser page after successful login.
Expected Result	User is authenticated, redirected to the dashboard, and the authenticated session persists (user remains logged in) after page refresh due to Redux Persist.
Actual Result	User was successfully authenticated and redirected to the dashboard. Session remained active after refresh, confirming correct Redux Persist rehydration.
Status	**PASS**

Test Case 2: Protected Route & Role-Based Access

Test Case ID	TC-02
Objective	To verify that role-protected routes correctly restrict access based on the logged-in user’s role, and that unauthorized access attempts are redirected appropriately.
Preconditions	User is logged in with a specific role (e.g., standard user, not admin); the Route Guard component is active on protected routes.
Test Steps	1. Log in as a user without administrative privileges. 2. Attempt to directly access a restricted/admin-only route via URL. 3. Log out and attempt to access any protected route while unauthenticated.
Expected Result	The unauthorized user is redirected away from the restricted route (e.g., to an unauthorized/dashboard page), and the unauthenticated user is redirected to the login page.
Actual Result	Access to the restricted route was correctly denied for the non-privileged role, and unauthenticated access attempts were correctly redirected to the login page as expected.
Status	**PASS**

Test Case 3: Procurement & Vendor Navigation

Test Case ID	TC-03
Objective	To verify that navigation between the Procurement module, Vendors list, and individual Vendor detail pages functions correctly.
Preconditions	User is logged in and authenticated; mock procurement and vendor data is loaded.
Test Steps	1. Navigate to the Procurement module from the main menu. 2. Navigate to the Vendors page. 3. Select a vendor from the vendor list. 4. Verify that the Vendor Detail page loads with the correct vendor's information. 5. Navigate back to the Vendors list.
Expected Result	Each navigation step correctly renders the corresponding page, the Vendor Detail page displays data matching the selected vendor, and back-navigation returns to the vendor list without errors.
Actual Result	All navigation transitions completed successfully. The Vendor Detail page correctly displayed data for the selected vendor, and back-navigation functioned as expected.
Status	**PASS**

Test Case 4: Redux State Management & Data Loading

Test Case ID	TC-04
Objective	To verify that Redux Toolkit slices correctly load, store, and reflect mock data across connected modules (e.g., dashboard, risk, compliance).
Preconditions	User is logged in; relevant modules (Dashboard, Risk, Compliance) are accessible.
Test Steps	1. Navigate to the Dashboard and observe chart/summary data loading. 2. Navigate to the Risk module and confirm risk register entries display correctly. 3. Navigate to the Compliance module and confirm compliance records display correctly. 4. Trigger a state-changing action (e.g., filter or status update) and confirm the UI updates accordingly.
Expected Result	Each module correctly retrieves and displays its corresponding mock data via its Redux slice, and any dispatched actions correctly update the UI in real time without requiring a page reload.
Actual Result	All modules correctly loaded and displayed data from their respective Redux slices. State-changing actions updated the UI immediately and consistently, confirming correct reducer behavior.
Status	**PASS**

Test Case 5: Form Validation and User Input

Test Case ID	TC-05
Objective	To verify that form input fields (e.g., vendor onboarding or procurement request forms) correctly validate user input using React Hook Form and Yup schema validation.
Preconditions	User is logged in and has navigated to a form-based screen (e.g., add vendor / new procurement request).
Test Steps	1. Attempt to submit the form with mandatory fields left empty. 2. Enter invalid data into a field with a defined format (e.g., an invalid email address). 3. Correct all fields with valid data and resubmit the form.
Expected Result	The form displays appropriate inline validation error messages for empty and invalid fields and prevents submission until all fields are valid. Upon correction, the form submits successfully.
Actual Result	Validation errors were correctly displayed for empty and invalid fields, and form submission was blocked until corrections were made. The form submitted successfully once valid data was entered.
Status	**PASS**

6 Test Summary

The table below summarizes the overall outcome of the testing effort conducted across all functional areas of the e-GRCP platform. This includes the five detailed manual test cases documented in Section 7, together with the full automated regression suite (Jest and React Testing Library) covering authentication, route guarding, Redux slices, and individual module pages.

Metric	Value
Total Automated Test Files	12
Total Test Cases Executed	32
Test Cases Passed	32
Test Cases Failed	0
Pass Percentage	100%

The breakdown of automated test cases by module is as follows:

Test File	Test Cases
RouteGuard.test.jsx	3
approvals.test.jsx	3
audit.test.jsx	2
auth.test.jsx	4
compliance.test.jsx	2
dashboard.test.jsx	3
procurement.test.jsx	3
reports.test.jsx	2
risk.test.jsx	2
settings.test.jsx	1
slices.test.js	5
vendor.test.jsx	2
Total	**32**

All executed test cases – covering authentication, role-based access control, navigation, Redux state management, Axios-based mock data communication, and form validation – passed successfully, with no critical or major defects identified during the final testing cycle.

7 Issues Observed

During the course of development and iterative testing, a small number of minor issues were identified. These were addressed and resolved prior to the final testing cycle documented in this report. No unresolved issues remain as of the final build.

- **Minor UI alignment adjustments:** Some Material UI grid components required spacing and alignment corrections on certain screens (e.g., vendor detail cards, dashboard summary tiles) to maintain visual consistency across screen sizes.
- **Route redirection improvements:** Early versions of the Route Guard occasionally redirected users to an incorrect fallback route after login; redirection logic was refined to correctly return users to their originally requested route.
- **Redux state synchronization refinements:** Minor timing inconsistencies were observed between mock data loading and initial component render in a small number of slices; loading state handling was refined to ensure the UI consistently reflects the correct state before rendering data-dependent components.

All of the issues listed above were identified during internal development testing, resolved by the development team, and re-verified before the final round of testing. No open issues were carried forward into the final release build.

8 Conclusion

The testing effort carried out on the Enterprise Governance, Risk, Compliance & Procurement Platform (e-GRCP) confirms that the application meets its intended functional requirements as a frontend enterprise platform. All 32 automated test cases across 12 test files, together with the five detailed manual scenarios documented in this report, passed successfully with a 100% pass rate and no outstanding defects.

Specifically, testing confirmed that:

- Routing and navigation, managed through React Router DOM, function correctly across all modules, including nested and role-protected routes.
- Authentication and role-based access control operate as designed, correctly restricting access based on user roles and maintaining session state through Redux Persist.
- Redux Toolkit state management functions reliably across all application slices, correctly reflecting data loaded from the mock JSON layer and responding correctly to dispatched actions.
- Axios-based communication with the mock data layer performs consistently, with data correctly retrieved and rendered across procurement, vendor, risk, compliance, and reporting modules.
- Form validation, implemented using React Hook Form and Yup, correctly enforces input rules and provides clear feedback to users.

Based on these results, the e-GRCP application is assessed to be functionally stable, structurally reliable, and maintainable in its current state. Its modular architecture – with clearly separated Redux slices, service layers, and route configuration – provides a solid foundation for future backend integration, at which point the current mock JSON data layer can be replaced with live API endpoints with minimal disruption to the existing frontend logic.

Overall, the application is considered ready to proceed to the next phase of development, including backend integration and expanded test coverage.